Symbolic Verification of ACAS-Xu Lookup Tables using Binary Decision Diagrams

Martin Boniol[1,2][0009−0005−0544−6778], Julien Brunel[2][0009−0004−3639−6681], Jean-Baptiste Chaudron[1][0000−0002−2142−1336], Christophe Garion[1][0000−0002−4467−2939], and Xavier Thirioux[1][0009−0002−1126−6835]

[1] Université de Toulouse, ISAE SUPAERO, Toulouse, France
[2] Université de Toulouse, DTIS, ONERA, Toulouse, France

Abstract. Airborne Collision Avoidance System Xu (ACAS-Xu) relies on large certified lookup tables (LUTs) that encode exact collision avoidance policies for unmanned aircraft. While these LUTs guarantee deterministic behavior, their size makes deployment and formal analysis difficult. In previous work, an exact symbolic compression of ACAS-Xu LUTs using Binary Decision Diagrams (BDDs) was introduced, enabling compact storage while preserving full decision semantics. Beyond compression, this symbolic representation provides a natural basis for formal verification. This paper extends the use of BDDs from exact representation to a unified framework for advanced verification of ACAS-Xu decision logic. First, we introduce relational and geometry-aware properties that capture meaningful operational behaviors beyond simple interval-based constraints, and show how they can be verified efficiently through symbolic BDD operations. Second, we extend verification from static state analysis to closed-loop dynamic reasoning by combining BDD-based decision logic with encounter simulation, enabling reachability-based detection of unsafe trajectories. Third, we propose an enriched BDD representation supporting multi-intruder scenarios, allowing formal reasoning over simultaneous conflict situations that cannot be handled by standard single-intruder encodings. Together, these contributions demonstrate that BDDs constitute not only an exact and compact replacement for ACAS-Xu LUTs, but also a scalable formal substrate for progressively richer verification, from symbolic states to dynamic trajectories and multi-agent encounters.

Keywords: Binary Decision Diagram · Formal Verification · ACAS-Xu · Embedded System · UAV.

1 Introduction

Ensuring safe separation between aircraft is a critical challenge in increasingly dense and heterogeneous airspace, particularly with the growing integration of unmanned aerial vehicles (UAVs). The Airborne Collision Avoidance System Xu (ACAS-Xu) addresses this challenge through a decision logic based on large pre-computed look-up tables (LUTs) [8,7], which encode optimal advisories derived

offline from Markov Decision Process (MDP) computations [12]. These LUTs constitute the certified operational core of the system and guarantee deterministic behavior over the discretized encounter space. Despite their operational reliability, ACAS-Xu LUTs present two major limitations. First, their very large memory footprint complicates deployment on embedded avionics platforms. Second, their explicit tabular form makes global reasoning about system behavior extremely difficult: while individual entries can be queried efficiently, verifying safety properties over the full state space remains computationally prohibitive. In previous work, we addressed these limitations by introducing an exact symbolic compression of ACAS-Xu LUTs using Binary Decision Diagrams (BDDs) [3,4]. This representation preserves the precise semantics of the original decision logic while drastically reducing memory usage, and provides a canonical Boolean structure amenable to formal reasoning. The key insight motivating the present work is that once ACAS-Xu LUTs are transformed into exact symbolic objects, they become more than compressed decision tables: they become a formal substrate on which increasingly expressive layers of verification can be built. In this sense, BDDs are not merely a compact representation mechanism, but the foundation of a unified verification framework for collision avoidance logic. A first level of verification consists in checking operational properties over symbolic encounter states. Prior analyses have mostly focused on simple interval-based constraints, where each input variable is bounded independently [10]. However, collision avoidance scenarios are inherently geometric: meaningful safety properties depend on relations between multiple parameters such as relative angle, heading, speed, and distance. Capturing such behaviors requires moving from simple box-constrained predicates to relational properties that encode structured encounter configurations. Yet verifying isolated symbolic states is not sufficient to characterize the behavior of an avoidance system operating in closed loop. ACAS-Xu decisions influence aircraft trajectories over time, and safety ultimately depends on the evolution of encounters under successive advisories. This introduces a second verification layer: moving from static state-based reasoning to dynamic trajectory-based reasoning, where symbolic decision logic must be combined with encounter dynamics to analyze reachability of unsafe situations such as collisions. Finally, even closed-loop single-intruder analysis remains only a partial view of the operational problem. Realistic airspace encounters often involve several simultaneous intruders, requiring ACAS-Xu to aggregate competing advisories across multiple conflict sources. Supporting such scenarios requires extending the symbolic representation itself, enriching BDD-encoded LUTs with score-aware structures capable of representing and verifying multi-intruder decision policies.

This paper develops this progressive extension of verification capabilities over ACAS-Xu LUTs. Starting from exact symbolic BDD representations, we first show how relational geometry-aware properties can be expressed and verified efficiently. We then extend verification to closed-loop dynamic simulations, enabling reachability-based analysis over encounter trajectories. Finally, we introduce an enriched BDD representation supporting multi-intruder reasoning

and formal verification of simultaneous conflict scenarios. Together, these contributions demonstrate that BDDs provide a unified and scalable framework for advancing ACAS-Xu verification from symbolic states to dynamic behaviors and multi-agent encounters.

In doing so, this work positions BDDs not only as an exact replacement for explicit LUT storage, but as a formal verification backbone capable of supporting progressively richer and more realistic analyses of certified collision avoidance systems.

2 Backgound and Verification Foundations

2.1 ACAS-Xu Decision Logic

ACAS-Xu is a collision avoidance system designed for unmanned aircraft within the ACAS-X family. Its role is to issue maneuver advisories that help an ownship avoid collisions with nearby intruder aircraft based on their relative geometry and motion. Unlike rule-based systems, ACAS-Xu relies on policies computed offline through MDP optimization, ensuring that advisories are selected according to formally optimized long-term safety criteria.

The decision logic operates over a discretized encounter state defined by six parameters, visible on Figure 1: time to loss of vertical separation (τ), horizontal distance (ρ), relative bearing angle (θ), relative heading angle (ψ), ownship speed (v_{own}), and intruder speed (v_{int}). Combined with the previous advisory a_{prev}, these variables determine the system input [7]. For each discretized state, ACAS-Xu stores precomputed advisory scores in large look-up tables (LUTs), from which the advisory with minimal expected cost is selected at runtime. In the horizontal avoidance setting considered here, five advisories are possible: Clear of Conflict (`COC`), Weak Left (`WL`), Weak Right (`WR`), Strong Left (`SL`), and Strong Right (`SR`) [7].

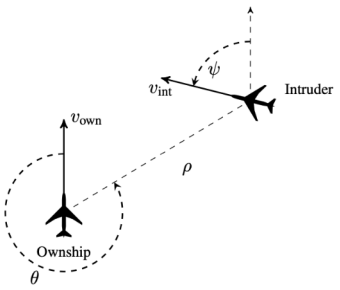


Fig. 1: ACAS-Xu geometry (taken from [])

The policy is implemented as several LUTs indexed by the previous advisory, each containing millions of entries. While this representation guarantees exact

and deterministic behavior, its size makes embedded deployment difficult and prevents efficient global verification, since reasoning over the full decision space requires exhaustive exploration of explicit table entries. This limitation motivates the use of symbolic representations better suited for compact storage and formal analysis [9].

2.2 Exact Symbolic Representation of ACAS-Xu LUTs with BDDs

To overcome the scalability limitations of explicit LUT storage, our previous work introduced an exact symbolic encoding of ACAS-Xu decision tables using BDDs [3]. Unlike neural-network-based compression methods, which approximate the original policy and may alter decision boundaries, BDDs preserve the exact semantics of the certified LUTs: every discretized encounter state is mapped to exactly the same advisory as in the original ACAS-Xu tables. The construction relies on encoding each discretized ACAS-Xu state as a Boolean vector obtained from the binary representation of its indexed parameters. These Boolean encodings are then used to build symbolic decision functions representing the advisory regions of the LUT. Because many neighboring states share identical advisories, the resulting BDDs exploit structural redundancies in the decision space and achieve substantial compression relative to explicit tabular storage. A key property of this representation is canonicity: under a fixed variable ordering, each advisory function admits a unique BDD form. This makes symbolic equivalence checking immediate and enables direct application of Boolean reasoning algorithms for verification. In practice, the resulting BDDs provide both compact memory usage and predictable evaluation latency, making them suitable for embedded deployment. Most importantly for the present work, this symbolic encoding transforms ACAS-Xu LUTs from passive data tables into manipulable formal objects. Once represented as BDDs, the decision logic can be combined directly with symbolic encodings of behavioral properties, enabling verification through standard Boolean operations such as conjunction, negation, and emptiness checking. This capability forms the foundation for the advanced verification layers developed in the following sections.

2.3 Existing Verification Approaches and Their Limitations

Formal verification of ACAS-Xu decision logic has so far mainly been explored through neural-network approximations of the original LUTs [5,14,13]. In these approaches, the tabular policy is first compressed into neural networks, and properties are then verified using dedicated solvers such as Reluplex [10]. This line of work demonstrated that safety-related properties can be checked over high-dimensional decision policies, but the verified properties are typically restricted to simple box-constrained input regions, where each state variable is bounded independently. While effective for proving local robustness conditions, such properties remain limited in expressiveness and do not capture the relational structure of realistic encounter geometries. In our previous work, the exact BDD encoding of ACAS-Xu LUTs enabled a first level of symbolic verification directly on the

certified decision logic, without approximation [3]. Because both the system and the properties are represented in the same Boolean formalism, verification reduces to conjunction and emptiness checks over decision diagrams. This already provides a major advantage over neural-network verification: reasoning is exact, counterexamples correspond directly to real LUT states, and no specialized solver beyond BDD operations is required. However, the properties considered so far remain largely similar in nature to those used in neural-network verification: they are primarily expressed as conjunctions of independent interval constraints over input variables. Such formulations are insufficient for capturing the geometric dependencies that characterize collision avoidance scenarios, where meaningful conditions often involve relations between several parameters (for example, coupled constraints between relative heading, bearing angle, and speed differences). More fundamentally, both neural-network and existing BDD-based approaches remain mostly limited to static state-level reasoning: they analyze isolated encounter states but cannot account for the temporal evolution of encounters under successive advisories. A second major limitation concerns operational scope. Current symbolic verification frameworks are restricted to single-intruder encounters, because standard ACAS-Xu BDD encodings represent only the final advisory decision and not the full score structure required to combine multiple simultaneous threats. As a result, they cannot directly model or verify multi-intruder conflict resolution, which is essential in realistic dense-airspace scenarios. These limitations motivate the progressive extensions developed in this paper: richer relational property verification over symbolic states, closed-loop dynamic verification over encounter trajectories, and enriched BDD representations supporting multi-intruder reasoning.

3 Verification of Geometry-Aware Properties

While exact BDD encodings make ACAS-Xu decision logic fully amenable to symbolic reasoning, their real advantage lies in the ability to express properties that reflect the geometric semantics of collision avoidance encounters. In practice, many meaningful operational requirements cannot be described solely by independent bounds on state variables. Collision scenarios are defined by structured spatial relationships: whether an intruder is converging, diverging, aligned, or crossing depends on combinations of distance, relative angles, and speed relations. Such situations correspond to families of encounter configurations governed by coupled constraints between parameters rather than isolated rectangular regions of the state space. These relational patterns are particularly important in ACAS-Xu because advisories are intended to reflect coherent maneuvering behavior over broad classes of geometrically similar encounters. Verifying only simple bounded regions may confirm local consistency, but does not establish whether the decision logic behaves correctly under generalized operational scenarios.

The objective of this section is therefore to lift verification from local interval-based checks to geometry-aware relational reasoning. By exploiting the symbolic

flexibility of BDDs, we define properties that capture encounter semantics at a higher level of abstraction and verify them directly over the exact ACAS-Xu decision logic.

3.1 Encoding Relational Geometric Properties

To illustrate the expressive power of symbolic BDD verification, we define a relational safety property capturing collision-course encounters. We consider encounter states in which the ownship and an intruder move along rectilinear constant-speed trajectories that intersect at a common future point. In such configurations, issuing a `COC` advisory is unsafe and should therefore never occur. Formally, the property states: *If both aircraft are predicted to reach the same future intersection point at the same time, ACAS-Xu must not output `COC`.* Unlike interval-based properties, this requirement cannot be expressed as independent bounds on state variables. It depends on coupled relations between relative distance, bearing angle, heading angle, and both aircraft velocities. As we see on the Figure 2, representing the situation of such property, a candidate collision configuration is modeled as a triangle whose vertices are: ownship current position, intruder current position and future intersection point. Its three sides are : ρ (current aircraft separation), d_{own} (ownship distance to intersection point) and d_{int} (intruder distance to intersection point).

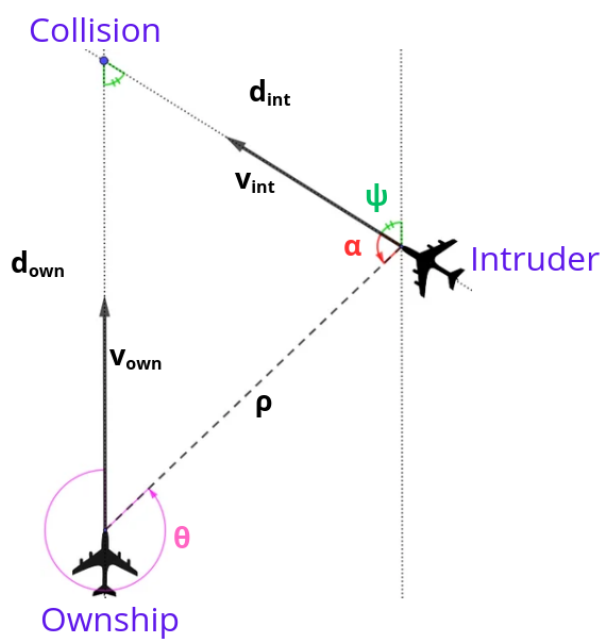


Fig. 2: Geometric proporty representation

Constraint 1 - Angular consistency: The collision angle α is computed independently in two ways. From encounter geometry and relative kinematics :

$$\alpha_{geom} = \theta - \psi - \pi \qquad \alpha_{trig} = \arccos\left(\frac{v_{int} - v_{own}\cos(\psi)}{\rho}\right)$$

To ensure consistency between observed encounter angles and motion geometry. We require : $|\alpha_{geom} - \alpha_{trig}| \leq \epsilon_\alpha$

Constraint 2 - Triangle closure: To guarantee that the three sides define a valid geometric triangle. The three distances must satisfy the law of cosines:

$$\rho^2 = d_{own}^2 + d_{int}^2 - 2d_{own}d_{int}\cos(\psi) \qquad d_{own} = \rho\frac{\sin(\theta - \psi - \pi)}{\sin(\psi)}$$

$$d_{int} = -\rho\frac{\sin(\theta)}{\sin(\psi)}$$

Constraint 3 - Temporal synchronization: A geometric intersection alone does not imply collision, both aircraft must reach the intersection simultaneously. Let : $t_{own} = \frac{d_{own}}{v_{own}}$ and $t_{int} = \frac{d_{int}}{v_{int}}$. A collision is retained only if: $|t_{own} - t_{int}| \leq \epsilon_t$

Taken together, these three constraints are sufficient to characterize a valid future collision:

- Constraint 1 guarantees angular coherence;
- Constraint 2 guarantees existence of a physically valid triangle;
- Constraint 3 guarantees simultaneous arrival at the common vertex.

Therefore, any retained state corresponds to two converging trajectories reaching the same future point at the same time, which exactly defines a collision-course encounter under the constant-velocity assumption.

All discretized states satisfying these conditions are encoded into a symbolic predicate C_{coll}. Verification then reduces to checking: $C_{coll} \wedge BDD_{ACAS} \wedge COC$. If this conjunction is empty, the property holds.

3.2 Verification Results

The verification of the relational collision predicate defined above was performed over the following operational domain:

- $\rho \in [499, 20000]$,
- $\tau \in [0, 100]$,
- $v_{own}, v_{int} \in [400, 1200]$

with geometric consistency thresholds: $\epsilon_\alpha = 0.01rad$, and $\epsilon_t = 0.45s$.

Under these settings, more than 1500 geometrically valid collision-course encounter states were identified and encoded into the predicate C_{coll}. The verification result is positive: no state satisfying these collision conditions produces a `COC`. Formally, $C_{coll} \wedge BDD_{ACAS} \wedge COC = False$, showing that ACAS-Xu never issues `COC` in the validated collision scenarios considered.

To evaluate the robustness of this result, we also varied the temporal synchronization threshold ϵ_t. When increasing the tolerance to $\epsilon_t = 0.5$, the property no longer holds. In this setting, 7 violations are identified among the 1740 evaluated scenarios. These counterexamples correspond to encounter configurations in which ACAS-Xu issues a `COC` advisory, while the ownship and the intruder reach the intersection point with a time difference of less than 0.5 seconds (e.g., 0.49 seconds). These counterexamples are particularly interesting because they lie near the boundary between safe path crossing and effective collision risk. Although such situations do not satisfy the stricter simultaneity criterion used in the verified property, they reveal encounter configurations that may still be operationally critical.

This result highlights an important advantage of relational symbolic verification: beyond establishing correctness properties, it enables a systematic exploration of safety margins and reveals borderline scenarios where the behavior of the decision logic may warrant further analysis.

4 Closed-Loop dynamic verification

The relational verification framework introduced in the previous section enables the analysis of geometrically meaningful encounter configurations at the level of individual states. While this already extends significantly the expressiveness of existing approaches, it ... remains inherently limited to static reasoning [2]. In practice, however, ACAS-Xu operates in a closed-loop setting: advisories issued at each decision step directly influence the future trajectory of the ownship, which in turn determines subsequent encounter states and decisions. As a result, safety cannot be assessed solely from isolated configurations, but must take into account the temporal evolution of encounters under successive advisories. In particular, situations that appear safe when considered locally may evolve into unsafe trajectories when the system is applied iteratively, and conversely, some locally critical states may be resolved through appropriate sequences of advisories. This highlights the need for a dynamic verification framework capable of reasoning over encounter trajectories rather than individual states.

In this section, we extend the BDD-based verification approach to a closed-loop setting by combining the symbolic decision logic with a physical model of aircraft motion. This enables the simulation of encounter evolution under ACAS-Xu advisories and allows safety properties to be evaluated through reachability analysis, such as detecting whether a collision can occur from a given set of initial conditions.

4.1 Closed-loop encounter model

We consider a simplified two-dimensional encounter model capturing the essential dynamics of horizontal conflict resolution, a standard point-mass kinematic model commonly used in aircraft conflict studies [11]. The ownship and the intruder are modeled as point masses moving in the horizontal plane under the following assumptions:

- motion is restricted to two dimensions ($\tau = 0$),
- both aircraft move at constant speeds v_{own} and v_{int},
- the intruder follows a straight-line trajectory with constant heading,
- the ownship can modify its heading according to ACAS-Xu advisories.

Let (x_o, y_o) and (x_i, y_i) denote the positions of the ownship and intruder, respectively. The ownship heading is denoted by ψ_o, while the intruder heading ψ_i remains constant.

State Representation: At each time step, the encounter state is expressed in relative polar coordinates:

$$\begin{aligned}
&\rho = \sqrt{(x_i - x_o)^2 + (y_i - y_o)^2} \\
&\theta = \operatorname{atan2}(x_i - x_o, y_i - y_o) - \psi_o \\
&\psi = \psi_i - \psi_o
\end{aligned}$$

Advisory-Drive Dynamics: At each decision step, the ACAS-Xu policy (encoded as a BDD) outputs an advisory $a \in \{COC, WR, WL, SR, SL\}$, which determines the angular rate of the ownship (with appropriate signs depending on turn direction) [Ref] :

$$\dot{\psi}_o = \begin{cases} 0 \quad (COC) \\ \omega_w = \pm 1.5 \;\; deg/s \quad (WR/WL) \\ \omega_s = \pm 3 \;\; deg/s \quad (SR/SL) \end{cases}$$

The ownship and intruder positions are then updated according to:

$$\begin{aligned}
x_o(t + \Delta t) &= x_o(t) + v_{own} \sin(\psi_o(t)) \Delta t \\
y_o(t + \Delta t) &= y_o(t) + v_{own} \cos(\psi_o(t)) \Delta t \\
x_i(t + \Delta t) &= x_i(t) + v_{int} \sin(\psi_i) \Delta t \\
y_i(t + \Delta t) &= y_i(t) + v_{int} \cos(\psi_i) \Delta t
\end{aligned}$$

Closed-Loop Execution: The system evolves in discrete decision steps of duration Δt, following a closed-loop structure:

1. compute relative state (ρ, θ, ψ),
2. query the ACAS-Xu BDD to obtain an advisory,
3. update the ownship heading accordingly,
4. propagate both aircraft positions using the motion equations.

This process is iterated until a predefined time horizon is reached.

Collision Criterion: A collision is detected whenever the relative distance falls below a fixed safety threshold: $\rho \leq \rho_{safe}$. In our experiments, we use $\rho_{safe} = 500$ feet.

4.2 Simulation framework

The closed-loop model described above is implemented as a discrete-time simulation combining symbolic decision logic with continuous motion dynamics, using a numerical integration framework based on the CODAC library [6]. At each control step, the current encounter state (ρ, θ, ψ) is first computed from the relative positions and headings of the aircraft. This state is then discretized according to the ACAS-Xu LUT grids, and the corresponding advisory is obtained by querying the BDD representation of the policy. The use of a BDD representation for the ACAS-Xu policy plays a key role in this process. It enables fast and exact evaluation of advisories for any discretized state, ensuring that the simulation faithfully reproduces the behavior of the underlying LUT without approximation or interpolation. The selected advisory determines the angular rate applied to the ownship, which is then used to update its heading. And the collision detection is performed continuously during the integration phase by monitoring the relative distance between the aircraft.

Between two decision steps, the motion of both aircraft is propagated using a finer time discretization in order to approximate continuous trajectories accurately. More precisely, the simulation follows a two-level time scheme: a control step (at which the advisory is updated) and a finer integration step (used to propagate the aircraft positions under constant velocities and angular rate). This separation allows the decision logic to operate at the same temporal resolution as the original ACAS-Xu system, while maintaining a sufficiently precise approximation of the underlying continuous dynamics.

The simulation proceeds iteratively by alternating between advisory computation and state propagation, starting from an initial encounter configuration and continuing until a predefined time horizon is reached or a collision is detected.

4.3 Reachability-based verification

The closed-loop simulation framework enables the analysis of encounter trajectories originating from sets of initial configurations, rather than individual states. To this end, we define families of initial conditions representing realistic encounter geometries. In particular, we consider sets of intruder positions distributed along parametric curves centered around the ownship, which remains initially located at the origin. Among these, we focus on elliptical encounter sets, which provide a structured and meaningful representation of potential conflict situations. For a given ellipse, the ownship is placed at its center, while the intruder is initialized at different positions along the curve. To model realistic interception scenarios, the intruder heading is chosen such that its trajectory is perpendicular to the ellipse at its initial position. This construction ensures

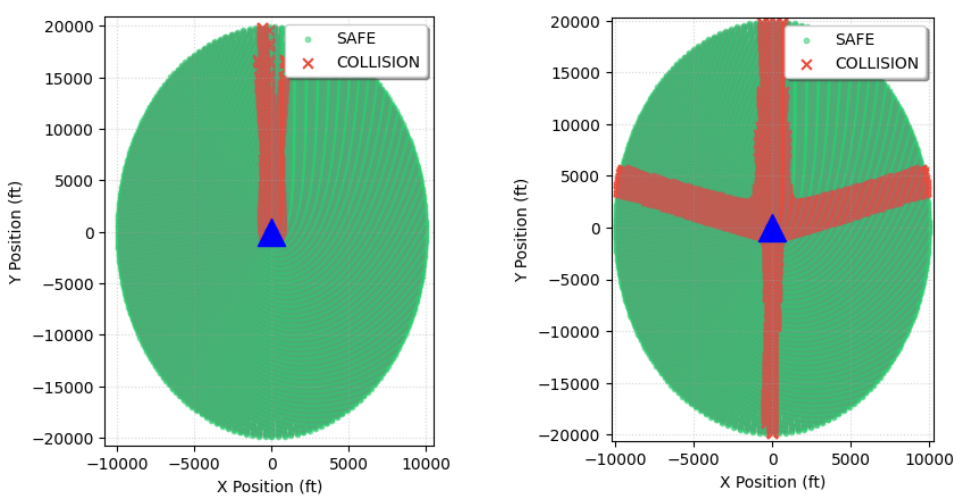
that the intruder is directed toward the interior of the ellipse, thereby generating encounter trajectories that intersect or approach the ownship path. By varying the size and shape of the ellipse, as well as the initial position along it, we obtain a diverse set of encounter configurations covering a wide range of geometrical situations. For each initial configuration, the closed-loop simulation is executed and the resulting trajectory is analyzed to determine whether a collision occurs. This leads to a reachability-based safety property, a classical notion in dynamical system verification [1]:

Is a collision reachable from a given set of initial states under ACAS-Xu LUTs advisories?

In this setting, safety is no longer evaluated at a single decision point, but over the entire trajectory induced by the interaction between the advisory logic and the system dynamics.

4.4 Experimental results

We evaluate the proposed closed-loop verification framework over families of encounter configurations generated from elliptical initial sets, as described in the previous section. For each configuration, a simulation is executed and labeled as *safe* or *collision* depending on whether the minimum separation threshold is violated. The results are visualized as heatmaps, where each point corresponds to an initial intruder position on the ellipse and its associated trajectory outcome.



(a) Heatmap closed-loop simulation $v_{own} = 702.4 > v_{int} = 100.0 \quad (ft/s)$

(b) Heatmap closed-loop simulation $v_{own} = 250.0 < v_{int} = 1097.5 \quad (ft/s)$

Fig. 3: Closed-Loop Simulation

We first compare two representative scenarios depending on the relative speeds of the aircraft. When the ownship is faster than the intruder, as it is in Figure 3a, the system exhibits a limited number of unsafe trajectories. However, the corresponding heatmap reveals a structured pattern: a thin but continuous region of collision cases appears. In contrast, when the intruder is faster than the ownship, the number of unsafe trajectories increases significantly, resulting in larger regions of the state space leading to collision, as shown in Figure 3b. This behavior is expected, as the ownship has reduced maneuvering capability to escape the encounter. A closer analysis of the collision cases in the first scenario reveals a particularly interesting phenomenon. The thin unsafe region corresponds to trajectories where the advisory alternates repeatedly between opposite maneuvers (e.g., WR/WL or SR/SL). This oscillatory behavior induces a zig-zag trajectory of the ownship, preventing it from committing to a consistent avoidance maneuver. As a result, the ownship fails to deviate sufficiently from the collision course and eventually encounters the intruder. These observations also provide insight into the role and limitations of the LUT-based policy representation. While the LUT encode the core decision logic of ACAS-Xu and satisfy the verified local safety properties (shown in Section 3), the closed-loop simulations reveal configurations in which this discretized policy alone does not ensure safe trajectory evolution. In particular, the emergence of oscillatory behaviors leading to collision highlights a gap between local correctness and global safety under closed-loop execution. This suggests that reasoning solely at the level of static decision mappings is insufficient to fully capture the safety properties of the system when deployed over time. From this perspective, the proposed simulation framework can be seen as a tool to isolate and analyze the intrinsic behavior of the LUT-based policy. It enables the identification of scenarios where the decision logic, when applied in isolation, fails to guarantee safe avoidance. It enables also the identification of underlying behavioral patterns responsible for these unsafe outcomes. The use of BDDs is instrumental in enabling this analysis. Unlike explicit LUT representations, which are difficult to manipulate and query exhaustively, the symbolic encoding provides a compact and structured representation of the decision logic. This allows efficient and exact evaluation of a large number of encounter states during simulation, while preserving full fidelity with respect to the original policy. Moreover, the same representation supports both the static relational verification of Section 3 and the dynamic closed-loop analysis presented here, providing a unified framework for reasoning about the system.

5 Multi-Intruder extension via score-aware BDD representation

The analyses developed so far rely on a simplified view of the ACAS-Xu policy, where each encounter state is associated with a single selected advisory. While this abstraction is sufficient for single-intruder reasoning, it hides an essential aspect of the underlying decision process: the relative ranking of candidate actions.

In ACAS-Xu, each action is associated with a score, and the selected advisory corresponds to the minimum-cost action [7,8]. In multi-intruder scenarios, decision making relies on comparing these scores across different encounters, typically combining them through a max–min aggregation scheme [7,8]. As a result, reasoning solely on the selected action is no longer sufficient to capture the behavior of the system. To address this limitation, we propose to enrich the BDD representation by incorporating score information into the symbolic model. Instead of encoding only the selected advisory, the BDD captures, for each state, both the selected action and the corresponding minimal score. This provides access to the relative ordering of decisions, which is the key ingredient required for multi-intruder reasoning. This extension is not intended to support full closed-loop simulation, but rather to enable static analysis of multi-intruder decision logic. In particular, it allows the formulation and verification of properties that depend on comparisons between scores associated with different intruders, thereby extending the scope of BDD-based verification beyond the single-intruder setting.

5.1 Score-Based BDD Encoding

To enable reasoning over action rankings, we construct an extended BDD representation that associates each encounter state with both a selected advisory and its corresponding score. More precisely, for each discretized state, we encode: (i) the action selected by the policy, and (ii) the minimal score associated with this action among all candidates. Since the number of distinct score values is finite, we encode scores using a binary representation over a fixed number of bits. In our implementation, 14 Boolean variables are sufficient to represent all distinct minimal scores. The BDD is structured as a set of subgraphs, each corresponding either to a score bit or to an action predicate. A set of selector variables is used to multiplex these subgraphs into a single unified structure. Each subgraph encodes the set of states for which the corresponding bit (or action) evaluates to true. This construction allows reconstructing, for any given state, both the selected action and the encoded score value. Importantly, since scores are ordered, their binary encoding can be directly used to compare actions across different states without requiring access to the original numerical values. This representation therefore provides a symbolic abstraction of the scoring mechanism underlying the ACAS-Xu decision process in multi-intruder situations, while remaining compatible with standard BDD manipulation techniques.

5.2 Static Verification Results for Multi-Intruder Scenarios

Using the score-aware BDD representation, we can extend the verification approach introduced in Section 3 to multi-intruder scenarios. We focus on properties defined over bounded regions of the input space, expressed as interval constraints on the encounter parameters. These properties follow the same structure as in the single-intruder case, but are evaluated independently for each intruder and then combined through score comparisons.

Given two sets of input constraints, one for each intruder, the verification procedure can be formalized as follows:

1. For each intruder, we compute the image of the score-based BDD under the corresponding input constraints. Formally, letting X_i denotes the input variables associated with intruder i and S the selector variables, this corresponds to: $I_i(S) = \exists X_i \left(B(X_i, S) \wedge C_i(X_i)\right)$
   where $C_i(X_i)$ denotes the input constraints and $B(X_i, S)$ the Boolean relation represented by the BDD. This yields a symbolic representation of all possible pairs (a, s), where a is a feasible advisory and s is the associated minimal score.
2. This results in two sets of candidate pairs, one per intruder, capturing all admissible decisions under the specified constraints: $\mathcal{P}_i = \{(a, s) \mid I_i(S)\}$
3. These sets are then combined by comparing scores across intruders. The final advisory is selected by choosing the action associated with the highest score among all candidate pairs:

$$(a^*, s^*) = \arg \max_{(a_i, s_i) \in \mathcal{P}_i,\ i \in \{1,2\}} s_i$$

   In other words, given two candidate pairs (a_1, s_1) and (a_2, s_2) obtained from each intruder, the selected advisory corresponds to the action associated with the maximal score.
   Depending on the property, we then check whether a given action is consistently preferred or dominated.

As an illustration, consider the following property that is formalized using interval constraints on both intruders ($i \in \{1, 2\}$):

If two intruders are sufficiently far away, the system should advise `COC`*.*

$$12000 \leq \rho_i \leq 62000, \quad 0.7 \leq \theta_i \leq \pi \ \vee \ -\pi \leq \theta_i \leq -0.7,$$
$$-\pi \leq \psi_i \leq -\pi + \epsilon, \quad 100 \leq v_{own,i} \leq 1200, \quad 0 \leq v_{int,i} \leq 1200,$$

For each intruder, we extract the set of feasible (a, s) pairs. The property holds if the score associated with `COC` is always greater than or equal to the scores of all alternative actions when combining both intruders, i.e., if `COC` is never dominated under the max–min aggregation. This approach preserves the simplicity of interval-based specifications while enabling reasoning over multi-intruder interactions. The use of BDDs ensures that all admissible states are considered symbolically, without explicit enumeration, and that score comparisons can be performed exactly over the resulting sets.

In addition to the illustrative property above, we evaluated two other interval-based properties capturing distinct multi-intruder configurations (e.g., symmetric encounters and clearly separated geometries). These properties follow the same verification procedure and differ only in the definition of the input constraints. Among these properties, two were successfully verified over the considered domains. In these cases, the expected advisory remains consistently preferred across all admissible combinations of intruder states, indicating that the

score-based decision logic behaves coherently when extended to multi-intruder settings. However, for the third one, the analysis reveals the existence of a counterexample. This result indicates that there exists at least one pair of admissible configurations for which the expected advisory is not selected after score aggregation. More importantly, it suggests that relying solely on LUT-based decision logic may lead to unexpected behaviors in certain multi-intruder situations. However, this observation must be interpreted with caution, as it may either reveal a limitation of the LUT-based policy or stem from an inadequately formulated property. The precise interpretation of this counterexample is still under investigation.

Overall, these results demonstrate that the proposed approach using BDDs enables the systematic analysis of multi-intruder decision logic and can effectively uncover non-trivial behaviors that are not observable in the single-intruder setting.

6 Conclusion and Discussion

This paper demonstrates how Binary Decision Diagrams (BDDs), when carefully constructed, can serve as a powerful tool for the verification of ACAS-Xu lookup tables. By encoding the advisory logic into a symbolic representation, we showed that both static and dynamic analyses can be performed efficiently over large portions of the state space. On the one hand, BDDs enable the verification of expressive state-based properties, including relational geometric constraints that cannot be captured by simple interval bounds. On the other hand, their integration within a closed-loop simulation framework allows us to analyze the temporal behavior of the system and to detect non-trivial patterns emerging from the interaction between discrete advisories and continuous dynamics. Through these analyses, we were able to identify several unexpected and potentially unsafe behaviors, such as configurations where aircraft reach a near-collision situation with very small time separation while the system still advises `COC`, or oscillatory advisory patterns leading to collision. These findings illustrate the ability of BDD-based verification to uncover subtle behaviors that are difficult to detect through standard testing approaches. Furthermore, the extension of the BDD representation to include action scores makes it possible to reason about more complex scenarios, such as multi-intruder encounters. This enables the verification of decision consistency properties beyond the single-intruder setting, further highlighting the flexibility of the approach. Importantly, the limitations observed in the lookup tables should not be seen as a limitation of the method itself, but rather as a direct outcome of applying systematic and exhaustive verification. In this sense, BDDs provide not only a verification framework, but also a tool for gaining deeper insight into the behavior of decision policies. We believe that this approach opens the way to more advanced verification strategies for collision avoidance systems, and we encourage further exploration of symbolic methods to analyze and validate complex decision logics.

References

1. Althoff, M.: An introduction to cora 2015. In: Frehse, G., Althoff, M. (eds.) ARCH14-15. 1st and 2nd International Workshop on Applied veRification for Continuous and Hybrid Systems. EPiC Series in Computing, vol. 34, pp. 120–151. EasyChair (2015). https://doi.org/10.29007/zbkv, /publications/paper/xMm
2. Alur, R.: Principles of Cyber-Physical Systems. MIT Press (2015)
3. Boniol, M., Brunel, J., Chaudron, J.B., Garion, C., Thirioux, X.: Compressing acas-xu lookup tables with binary decision diagrams. In: NASA Formal Methods (NFM). Springer (2026). https://doi.org/10.1007/978-3-030-83903-1_3
4. Bryant, R.E.: Graph-based algorithms for boolean function manipulation. IEEE Transactions on Computers (1986)
5. Clavière, A.: Safety Verification of Neural Network Based Systems Using Formal Methods. Ph.D. thesis, ISAE-SUPAERO (2023), http://www.theses.fr/2023ESAE0041/document, thèse de doctorat, Toulouse. Réf. 2023ESAE0041
6. Desrochers, B., Jaulin, L.: Codac: A python library for constraint programming and interval analysis (2023), https://codac.io
7. EUROCAE: ED-275: Minimum operational performance standards for airborne collision avoidance system ACAS-Xu. Tech. rep., EUROCAE (Dec 2020), https://www.eurocae.net/documents-and-projects/documents/ed-275/, licensed to ONERA
8. EUROCONTROL: ACAS guide (2022), https://www.eurocontrol.int/system/acas
9. Julian, K.D., Kochenderfer, M.J., Owen, M.P.: Deep neural network compression for aircraft collision avoidance systems. Journal of Guidance, Control, and Dynamics **42**(3), 598–608 (2019). https://doi.org/10.2514/1.G003724
10. Katz, G., Barrett, C., Dill, D.L., Julian, K., Kochenderfer, M.J.: Reluplex: An efficient SMT solver for verifying deep neural networks. In: Majumdar, R., Kunčak, V. (eds.) Computer Aided Verification. pp. 97–117. Springer (2017). https://doi.org/10.1007/978-3-319-63387-9_5
11. Kochenderfer, M.J.: Decision Making Under Uncertainty: Theory and Application. MIT Press (2015)
12. Kochenderfer, M.J., Chryssanthacopoulos, J.P.: Robust airborne collision avoidance through dynamic programming. Tech. Rep. ATC-371, MIT Lincoln Laboratory, Lexington, MA (Jan 2011), https://apps.dtic.mil/sti/pdfs/ADA537522.pdf, prepared for the Federal Aviation Administration
13. Lopez, J., Johnson, T., Tran, H.D., Johnson, T.T.: Verification of neural network compression of ACAS-Xu lookup tables with star set reachability. In: Assured Autonomy Workshop (2021), https://assured-autonomy.org/publications/verification-neural-network-compression-acas-xu-lookup-tables-star-set-reachability
14. Sheikhi, S., Bak, S.: Closed-loop ACAS-Xu neural network verification. In: Applied Verification for Continuous and Hybrid Systems (ARCH) (2023), https://easychair.org/publications/paper/mmSLW